



Advanced DBMS and Administration

UNIT 1

Rational Database Design:

Relational Model in DBMS

The relational data model was introduced by C. F. Codd in 1970. Currently, it is the most widely used data model. The relational data model describes the world as “a collection of inter-related relations (or tables).” A relational data model involves the use of data tables that collect groups of elements into relations. These models work based on the idea that each table setup will include a primary key or identifier. Other tables use that identifier to provide "relational" data links and results.

Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

Domain: It contains a set of atomic values that an attribute can take.

Attribute: It contains the name of a column in a particular table. Each attribute A_i must have a domain, $dom(A_i)$

Relational instance: In the relational database system, the relational instance is represented by a finite set of tuples. Relation instances do not have duplicate tuples.

Relational schema: A relational schema contains the name of the relation and name of all columns or attributes.

Relational key: In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

Example: STUDENT Relation

NAME	ROLL_NO	PHONE_NO	ADDRESS	AGE
Ram	14795	7305758992	Noida	24
Shyam	12839	9026288936	Delhi	35
Laxman	33289	8583287182	Gurugram	20
Mahesh	27857	7086819134	Ghaziabad	27
Ganesh	17282	9028 9i3988	Delhi	40

- In the given table, NAME, ROLL_NO, PHONE_NO, ADDRESS, and AGE are the attributes.
- The instance of schema STUDENT has 5 tuples.
- t3 = <Laxman, 33289, 8583287182, Gurugram, 20>

Properties of Relations

- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value
- Each attribute contains a distinct name
- Attribute domain has no significance
- tuple has no duplicate value
- Order of tuple can have a different sequence

There are four stages of a Relational Database Model which are as follows –

- **Relations and attributes** – The various tables and attributes related to each table are identified. The tables represent entities, and the attributes represent the properties of the respective entities.
- **Primary keys** – The attribute or set of attributes that help in uniquely identifying a record is identified and assigned as the primary key.
- **Relationships** – The relationships between the various tables are established with the help of foreign keys. Foreign keys are attributes occurring in a table that are primary keys of another table. The types of relationships that can exist between the relations (tables) are One to one, One to many, and Many to many
- **Normalization** – This is the process of optimizing the database structure. Normalization simplifies the database design to avoid redundancy and confusion. The different normal forms are as follows:
 1. First normal form
 2. Second normal form
 3. Third normal form
 4. Boyce-Codd normal form
 5. Fifth normal form

By applying a set of rules, a table is normalized into the above normal forms in a linearly progressive fashion. The efficiency of the design gets better with each higher degree of normalization.

Functional Dependency

The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table.

1. $X \rightarrow Y$

The left side of FD is known as a determinant, the right side of the production is known as a dependent.

Consider a relation with four attributes A, B, C and D,

R (ABCD)

1. $A \rightarrow BCD$
2. $B \rightarrow CD$

- For the first functional dependency $A \rightarrow BCD$, attributes B, C and D are functionally dependent on attribute A.
- Function dependency $B \rightarrow CD$ has two attributes C and D functionally depending upon attribute B.

Example:

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

From the above table we can conclude some valid functional dependencies:

- $\text{roll_no} \rightarrow \{\text{name}, \text{dept_name}, \text{dept_building}\}$, $\rightarrow$ Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency
- $\text{roll_no} \rightarrow \text{dept_name}$, Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.

- dept_name $\rightarrow$ dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building
- More valid functional dependencies: roll_no $\rightarrow$ name, {roll_no, name} $\twoheadrightarrow$ {dept_name, dept_building}, etc.

Inference Rule (IR):

- The Armstrong's axioms are the basic inference rule.
- Armstrong's axioms are used to conclude functional dependencies on a relational database.
- The inference rule is a type of assertion. It can apply to a set of FD(functional dependency) to derive other FD.
- Using the inference rule, we can derive additional functional dependency from the initial set.

The Functional dependency has 6 types of inference rule:

1. Reflexive Rule (IR₁)

In the reflexive rule, if Y is a subset of X, then X determines Y.

1. If $X \supseteq Y$ then $X \rightarrow Y$

Example:

1. $X = \{a, b, c, d, e\}$
2. $Y = \{a, b, c\}$

2. Augmentation Rule (IR₂)

The augmentation is also called as a partial dependency. In augmentation, if X determines Y, then XZ determines YZ for any Z.

1. If $X \rightarrow Y$ then $XZ \rightarrow YZ$

Example:

1. For R(ABCD), if $A \rightarrow B$ then $AC \rightarrow BC$

3. Transitive Rule (IR₃)

In the transitive rule, if X determines Y and Y determine Z, then X must also determine Z.

1. If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

4. Union Rule (IR_4)

Union rule says, if X determines Y and X determines Z, then X must also determine Y and Z.

1. If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$

Proof:

1. $X \rightarrow Y$ (given)
2. $X \rightarrow Z$ (given)
3. $X \rightarrow XY$ (using IR_2 on 1 by augmentation with X. Where $XX = X$)
4. $XY \rightarrow YZ$ (using IR_2 on 2 by augmentation with Y)
5. $X \rightarrow YZ$ (using IR_3 on 3 and 4)

5. Decomposition Rule (IR_5)

Decomposition rule is also known as project rule. It is the reverse of union rule.

This Rule says, if X determines Y and Z, then X determines Y and X determines Z separately.

1. If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$

Proof:

1. $X \rightarrow YZ$ (given)
2. $YZ \rightarrow Y$ (using IR_1 Rule)
3. $X \rightarrow Y$ (using IR_3 on 1 and 2)

6. Pseudo transitive Rule (IR_6)

In Pseudo transitive Rule, if X determines Y and YZ determines W, then XZ determines W.

1. If $X \rightarrow Y$ and $YZ \rightarrow W$ then $XZ \rightarrow W$

Proof:

1. $X \rightarrow Y$ (given)
2. $WY \rightarrow Z$ (given)
3. $WX \rightarrow WY$ (using IR_2 on 1 by augmenting with W)
4. $WX \rightarrow Z$ (using IR_3 on 3 and 2)

Types of Functional Dependencies in DBMS

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency
5. Fullyb functional dependency

1. Trivial Functional Dependency

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is the **subset of X** , then it is called trivial functional dependency

Example:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\{\text{roll_no, name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set $\{\text{roll_no, name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

2. Non-trivial Functional Dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is **not a subset of X** , then it is called Non-trivial functional dependency.

Example:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent **name** is **not a subset** of determinant **roll_no**. Similarly, $\{\text{roll_no, name}\} \rightarrow \text{age}$ is also a non-trivial functional dependency, since **age** is **not a subset of $\{\text{roll_no, name}\}$**

3. Multivalued Functional Dependency

In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other**. i.e. If $a \rightarrow \{b, c\}$ and there exists **no functional dependency** between **b** and **c**, then it is called a **multivalued functional dependency**.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

Here, $\text{roll_no} \rightarrow \{\text{name}, \text{age}\}$ is a multivalued functional dependency, since the dependents **name** & **age** are **not dependent** on each other (i.e. $\text{name} \rightarrow \text{age}$ or $\text{age} \rightarrow \text{name}$ doesn't exist)

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$. This is a **transitive functional dependency**.

For example,

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, $\text{enrol_no} \rightarrow \text{dept}$ and $\text{dept} \rightarrow \text{building_no}$. Hence, according to the axiom of transitivity, $\text{enrol_no} \rightarrow \text{building_no}$ is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

5. Fully Functional Dependency

In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes. If a relation R has attributes X, Y, Z with the dependencies $X \rightarrow Y$ and $X \rightarrow Z$ which states that those dependencies are fully functional.

Ex:

(A, B, C) -----> D

Enrol_no , name $\rightarrow$ building_no.

Enrol_no $\rightarrow$ building_no

name $\rightarrow$ building_no

What is a Candidate Key

A candidate key is a subset of a super key set where the key which contains no redundant attribute is none other than a **Candidate Key**. In order to select the candidate keys from the set of super key, we need to look at the super key set.

A **candidate key** is a part of a key known as **Super Key**, where the super key is the super set of all those attributes that can uniquely identify a table.

Role of a Candidate Key

The role of a candidate key is to identify a table row or column uniquely. Also, the value of a candidate key cannot be Null. The description of a candidate key is "no redundant attributes" and being a "minimal representation of a tuple," according to the Experts.

How a Candidate key is different from a Primary Key

Although the purpose of both candidate and the primary key is the same, that is to uniquely identify the tuples, and then also they are different from each other. It is because, in a table, we can have one or more than one candidate key, but we can create only one primary key for a table. Thus, from the number of obtained candidate keys, we can identify the appropriate primary key. However, if there is only one candidate key in a table, then it can be considered for both key constraints.

Example:

STUD_NO is the candidate key for relation STUDENT.

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

- ✓ The candidate key can be simple (having only one attribute) or composite as well.

Example:

{STUD_NO, COURSE_NO} is a composite candidate key for relation STUDENT_COURSE.

Table STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

Normalization

Normal Forms Based on Primary Keys

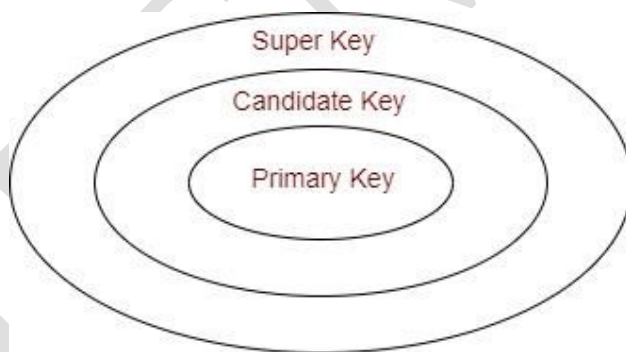
Normalization is a process of organizing data in a database to reduce redundancy and improve data consistency.

Primary keys are really important in organizing information in a database.

They help to make sure that every row in a table has a unique identification so that nothing gets mixed up or lost.

Types of Keys used in Normalization

There are several types of keys used in normalization in database management systems (DBMS). These are explained as follows.



Super Key

A super key is a set of one or more attributes that uniquely identifies each record in a table. A super key may contain more attributes than necessary to uniquely identify each record.

Candidate Key

A candidate key is a minimal super key that can uniquely identify each record in a table. In other words, it is a super key that does not contain any unnecessary attributes.

Primary Key

A primary key is a candidate key that has been selected to uniquely identify each record in a table. It is used to enforce entity integrity, and is typically denoted by an underline or a key symbol.

Alternate Key

An alternate key is a candidate key that is not selected to be the primary key. It is used to enforce uniqueness, and may be used as a reference key in another table.

Foreign Key

A foreign key is a key that is used to link two tables together. It is a column (or set of columns) in one table that refers to the primary key of another table.

Normalization is the process of removing redundancy from database and to avoid insertion, updation and deletion anomalies.

Data modification anomalies can be categorized into three types:

- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Types of Normal Forms:

First Normal Form (1NF)

1NF requires that each column in a table contains atomic values and that each row is uniquely identified. This means that a table cannot have repeating groups or arrays as columns, and each row must have a unique primary key.

Example

A table is in 1NF if each column contains atomic values and each row is uniquely identified. For example, a table that lists customers and their phone numbers –

Customer ID	Name	Phone Numbers
1	John	555-1234, 555-5678
2	Jane	555-9876
3	Michael	555-5555

This violates 1NF because the Phone Numbers column contains repeating groups.

To normalize this table to 1NF, we can split the Phone Numbers column into separate rows and add a separate primary key column –

Customer ID	Name	Phone Number
1	John	555-1234
1	John	555-5678
2	Jane	555-9876
3	Michael	555-5555

Second Normal Form (2NF)

2NF builds on 1NF by requiring that each non-primary key column in a table is fully functionally dependent on the primary key. This means that a table should not have partial dependencies, where a non-primary key column depends on only part of the primary key.

Example

A table is in 2NF if each non-primary key column is fully functionally dependent on the primary key. For example, a table that lists orders and their line items:

Order ID	Customer ID	Customer Name	Item ID	Item Name	Quantity
1	1	John	1	Shirt	2
1	1	John	2	Pants	1
2	2	Jane	1	Shirt	1
2	2	Jane	3	Hat	3

This violates 2NF because the Customer Name column depends on only part of the primary key (Customer ID). To normalize this table to 2NF, we can split it into two tables –

Order ID	Customer ID	Item ID	Quantity
1	1	1	2
1	1	2	1
2	2	1	1
2	2	3	3
Customer ID		Customer Name	
1		John	
2		Jane	

Third Normal Form (3NF)

3NF builds on 2NF by requiring that each non-primary key column in a table is not transitively dependent on the primary key. This means that a table should not have transitive dependencies, where a non-primary key column depends on another non-primary key column.

Example

To explain 3NF further, let's consider an example of a table that lists customer orders –

Order ID	Customer ID	Customer Name	Customer City	Order Date	Order Total
1	100	John Smith	New York	2022-01-01	100
2	101	Jane Doe	Los Angeles	2022-01-02	200
3	102	Bob Johnson	San Francisco	2022-01-03	300

In this example, the non-primary key column "Customer City" is transitively dependent on the primary key. That is, it depends on "Customer ID", which is not part of the primary key, instead of depending directly on the primary key "Order ID". To bring this table to 3NF, we can split it into two tables.

Table 1: Customers

Customer ID	Customer Name	Customer City
100	John Smith	New York
101	Jane Doe	Los Angeles
102	Bob Johnson	San Francisco

Table 2: Orders

Order ID	Customer ID	Order Date	Order Total
1	100	2022-01-01	100
2	101	2022-01-02	200
3	102	2022-01-03	300

Now, the "Customer City" column is no longer transitively dependent on the primary key and is instead in a separate table that has a direct relationship with the primary key. This makes the table 3NF-compliant.

Boyce-Codd Normal Form (BCNF)

BCNF is a stricter form of 3NF that applies to tables with more than one candidate key. BCNF requires that each non-trivial dependency in a table is a dependency on a candidate key. This means that a table should not have non-trivial dependencies, where a non-primary key column depends on another non-primary key column. BCNF ensures that each table in a database is a separate entity and eliminates redundancies.

Example

A table is in BCNF if each determinant is a candidate key. In other words, every non-trivial functional dependency in the table must be on a candidate key. For example, consider a table that lists information about books and their authors –

Table: Books

Book ID	Title	Author ID	Author Name	Author Nationality
1	Crime and Punishment	100	Fyodor Dostoevsky	Russian
2	The Great Gatsby	101	F. Scott Fitzgerald	American

3	Pride and Prejudice	102	Jane Austen	British
---	---------------------	-----	-------------	---------

In this example, the functional dependency between "Author ID" and "Author Name" violates BCNF because it is not on a candidate key. To bring this table to BCNF, we can split it into two tables –

Table 1: Authors

Author ID	Author Name	Author Nationality
101	Fyodor Dostoevsky	Russian
101	F. Scott Fitzgerald	American
102	Jane Austen	British

Table 2: Books

Book ID	Title	Author ID
1	Crime and Punishment	100
2	The Great Gatsby	101
3	Pride and Prejudice	102

Now, the "Author Name" and "Author Nationality" columns are not transitively dependent on the primary key, and the table is in BCNF.

Fourth Normal Form (4NF)

4NF builds on BCNF by requiring that a table should not have multi-valued dependencies. A multi-valued dependency occurs when a non-primary key column depends on a combination of other non-primary key columns. For example, a table that lists customer orders with a primary key of order ID and non-primary key columns for customer ID and order items violates 4NF because order items depend on both order ID and customer ID.

For example, a table that lists orders and their products, with columns for order ID, product ID, and product details, violates 4NF because the product details depend on the combination of order ID and product ID.

Example

Consider the following table of orders and products

Order ID	Product ID	Product Name	Product Description
1	100	Widget	Red Widget
1	200	Widget	Blue Widget
2	100	Widget	Red Widget
2	300	Thing	Green Thing
3	200	Widget	Blue Widget
3	300	Thing	Green Thing

In this table, the product name and description depend on both the order ID and product ID, creating a multi-valued dependency. To bring the table into 4NF, we can split it into three tables –

Order ID	Product ID
1	100
1	200
2	100
2	300
3	200
3	300
Product ID	Product Name
100	Widget
200	Widget
300	Thing

Product ID	Product Description
100	Red Widget
200	Blue Widget
300	Green Thing

Fifth Normal Form / Projected Normal Form (5NF)

A relation R is in [Fifth Normal Form](#) if and only if every non-trivial join dependency in R is implied by the candidate keys of R. A relation decomposed into two relations must have [lossless join](#) Property, which ensures that no spurious or extra tuples are generated when relations are reunited through a natural join.

Properties

A relation R is in 5NF if and only if it satisfies the following conditions:

1. R should be already in 4NF.
2. It cannot be further non loss decomposed (join dependency).

Example – Consider the above schema, with a case as “if a company makes a product and an agent is an agent for that company, then he always sells that product for the company”. Under these circumstances, the ACP table is shown as:

Table ACP

Agent	Company	Product
A1	PQR	Nut
A1	PQR	Bolt
A1	XYZ	Nut
A1	XYZ	Bolt
A2	PQR	Nut

The relation ACP is again decomposed into 3 relations. Now, the natural Join of all three relations will be shown as:

Table R1

Agent	Company
A1	PQR

Agent	Company
A1	XYZ
A2	PQR

Table R2

Agent	Product
A1	Nut
A1	Bolt
A2	Nut

Table R3

Company	Product
PQR	Nut
PQR	Bolt
XYZ	Nut
XYZ	Bolt

The result of the Natural Join of R1 and R3 over 'Company' and then the [Natural Join](#) of R13 and R2 over 'Agent' and 'Product' will be **Table ACP**.

Hence, in this example, all the redundancies are eliminated, and the decomposition of ACP is a lossless join decomposition. Therefore, the relation is in 5NF as it does not violate the property of [lossless join](#).

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transitive dependency exists.

BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dep
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should

Lossless join decomposition

Lossless-join decomposition is a process in which a relation is decomposed into two or more relations. This property guarantees that the extra or less tuple generation problem does not occur and no information is lost from the original relation during the decomposition. It is also known as non-additive join decomposition.

When the sub relations combine again then the new relation must be the same as the original relation was before decomposition.

Consider a relation R if we decomposed it into sub-parts relation R1 and relation R2.

The decomposition is lossless when it satisfies the following statement –

- If we union the sub Relation R1 and R2 then it must contain all the attributes that are available in the original relation R before decomposition.
- Intersections of R1 and R2 cannot be Null. The sub relation must contain a common attribute. The common attribute must contain unique data.

The common attribute must be a super key of sub relations either R1 or R2.

Here,

$R = (A, B, C)$

$R1 = (A, B)$

$R2 = (B, C)$

The relation R has three attributes A, B, and C. The relation R is decomposed into two relation R1 and R2. . R1 and R2 both have 2-2 attributes. The common attributes are B.

The Value in Column B must be unique. if it contains a duplicate value then the Lossless-join decomposition is not possible.

Draw a table of Relation R with Raw Data –

R (A, B, C)

A	B	C
12	25	34
10	36	09
12	42	30

It decomposes into the two sub relations –

R1 (A, B)

A	B
12	25
10	36
12	42

R2 (B, C)

B	C
25	34
36	09
42	30

Now, we can check the first condition for Lossless-join decomposition.

The union of sub relation R1 and R2 is the same as relation R.

$R_1 \cup R_2 = R$

We get the following result –

A	B	C
12	25	34
10	36	09
12	42	30

The relation is the same as the original relation R. Hence, the above decomposition is Lossless-join decomposition.

Dependency Preserving Decomposition

- It is an important constraint of the database.
- In the dependency preservation, at least one decomposed table must satisfy every dependency.
- If we decompose a relation R into relations R1 and R2, all dependencies of R must be part of **either R1 or R2 or must be derivable from combination of functional dependencies(FD) of R1 and R2**
- Suppose a relation R(A,B,C,D) with FD set **{A→BC}** is decomposed into R1(ABC) and R2(AD) which is dependency preserving because **FD A→BC is a part of R1(ABC)**

Example

Consider a schema **R(A,B,C,D)** and functional dependencies **A→B and C→D** which is decomposed into **R1(AB) and R2(CD)**

This decomposition is **dependency preserving decomposition** because

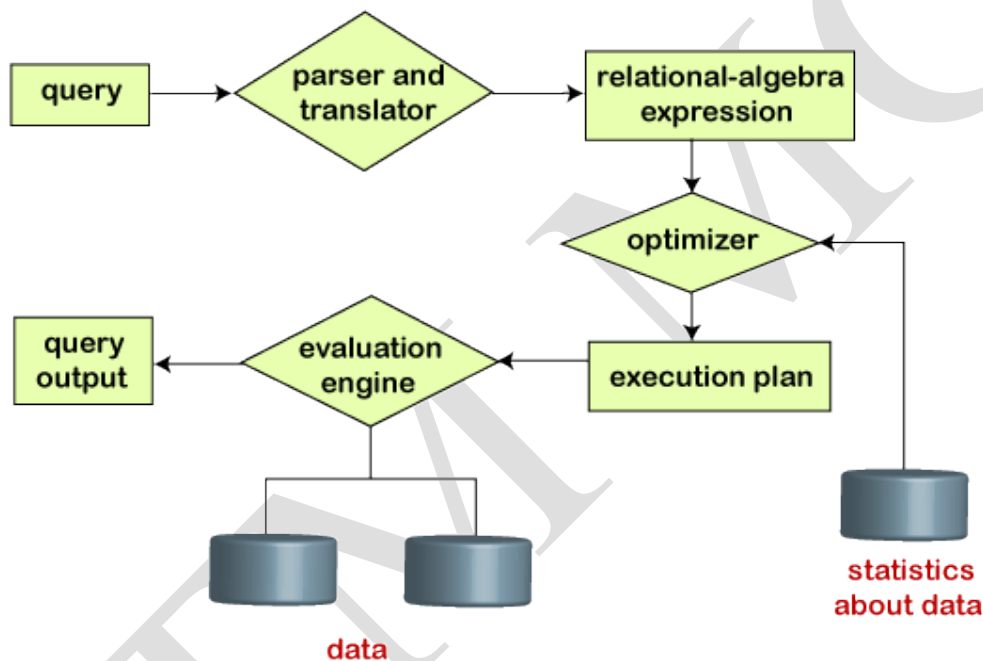
- **A→B** can be ensured in **R1(AB)**
- **C→D** can be ensured in **R2(CD)**

Query Processing

Query Processing is a translation of high-level queries into low-level expression. It is a step wise process that can be used at the physical level of the file system, query optimization and actual execution of the query to get the result.

It requires the basic concepts of relational algebra and file structure. It refers to the range of activities that are involved in extracting data from the database. It includes translation of queries in high-level database languages into expressions that can be implemented at the physical level of the file system.

Given below is the diagram that shows the steps of the structured query language (SQL) query processing –



Steps in query processing

In query processing, it takes various steps for fetching the data from the database. The steps involved are:

1. Parsing and translation
2. Optimization
3. Evaluation

1. Parsing and translation

Translate the query into its internal form. This is then translated into relational algebra.

Parser checks syntax, verifies relation.

2. Optimization

- SQL is a very high level language:
 - The users specify what to search for- not how the search is actually done
 - The algorithms are chosen automatically by the DBMS.
- For a given SQL query there may be many possible execution plans.
- Amongst all equivalent plans choose the one with lowest cost.
- Cost is estimated using statistical information from the database catalog.

3. Evaluation

- The query evaluation engine takes a query evaluation plan, executes that plan and returns the answer to that query.

Query Interpretation

Query interpretation in the context of databases (DBMS - Database Management System) refers to the process of understanding and translating a user's query or request into a form that can be executed against the database to retrieve the desired information.

Here's a breakdown of the steps involved in query interpretation within a DBMS:

- Parsing:** The first step is parsing the query to ensure it follows the syntax and structure of the query language (e.g., SQL). This involves checking for proper keywords, clauses, and punctuation.
- Semantic Analysis:** After parsing, the system performs semantic analysis to verify that the query's structure and elements are meaningful and valid. This step checks for things like the existence of specified tables and columns.
- Query Optimization:** Once the query is parsed and validated, the system may optimize the query to enhance performance. This involves selecting the most efficient way to execute the query, considering factors such as indexing, join strategies, and data access paths.
- Query Execution Plan:** Based on the optimized query, the system generates a query execution plan. This plan outlines the steps needed to retrieve the required data, including the order of operations (e.g., joins, filtering, sorting) and the chosen algorithms.
- Execution:** The actual execution of the query takes place according to the generated execution plan. The system interacts with the database to fetch and process the necessary data.
- Result Generation:** After executing the query, the DBMS processes the retrieved data according to the specified operations (e.g., aggregations, projections) and formats the result in a suitable output format.

7. **Result Presentation:** Finally, the DBMS presents the results to the user in a readable format, which could be a table, a graph, or any other suitable form based on the query and application requirements.

Effective query interpretation and optimization are crucial for efficient database operations, ensuring that queries are processed in the most efficient and timely manner, especially in large-scale databases with complex data structures.

Transformation of Relational Expressions

Two relational-algebra expressions are said to be equivalent if, on every legal database instance, the two expressions generate the same set of tuples.

Note that the order of the tuples is irrelevant; the two expressions may generate the tuples in different orders, but would be considered equivalent as long as the set of tuples is the same.

The first step of the optimizer says to implement such expressions that are logically equivalent to the given expression. For implementing such a step, we use the equivalence rule that describes the method to transform the generated expression into a logically equivalent expression.

Equivalence Rules

The equivalence rule says that expressions of two forms are the same or equivalent because both expressions produce the same outputs on any legal database instance. It means that we can possibly replace the expression of the first form with that of the second form and replace the expression of the second form with an expression of the first form. Thus, the optimizer of the query-evaluation plan uses such an equivalence rule or method for transforming expressions into the logically equivalent one.

The optimizer uses various equivalence rules on relational-algebra expressions for transforming the relational expressions. For describing each rule, we will use the following symbols:

$\theta, \theta_1, \theta_2 \dots$: Used for denoting the predicates.

$L_1, L_2, L_3 \dots$: Used for denoting the list of attributes.

$E, E_1, E_2 \dots$: Represents the relational-algebra expressions.

Rule 1: Cascade of σ

This rule states the deconstruction of the conjunctive selection operations into a sequence of individual selections. Such a transformation is known as a **cascade of σ** .

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

Rule 2: Commutative Rule

a) This rule states that selections operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

b) Theta Join (θ) is commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1 \text{ (θ is in subscript with the join symbol)}$$

However, in the case of theta join, the equivalence rule does not work if the order of attributes is considered. Natural join is a special case of Theta join, and natural join is also commutative.

However, in the case of theta join, the equivalence rule does not work if the order of attributes is considered. Natural join is a special case of Theta join, and natural join is also commutative.

Rule 3: Cascade of Π

This rule states that we only need the final operations in the sequence of the projection operations, and other operations are omitted. Such a transformation is referred to as a **cascade of Π** .

$$\Pi_{L1}(\Pi_{L2}(\dots(\Pi_{Ln}(E))\dots)) = \Pi_{L1}(E)$$

Rule 4: We can combine the selections with Cartesian products as well as theta joins

Rule 4: We can combine the selections with Cartesian products as well as theta joins

1. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
2. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

Rule 5: Associative Rule

a) This rule states that natural join operations are associative.

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

b) Theta joins are associative for the following expression:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

In the theta associativity, θ_2 involves the attributes from E_2 and E_3 only. There may be chances of empty conditions, and thereby it concludes that Cartesian Product is also associative.

Rule 6: Distribution of the Selection operation over the Theta join.

Under two following conditions, the selection operation gets distributed over the theta-join operation:

a) When all attributes in the selection condition θ_0 include only attributes of one of the expressions which are being joined.

$$\sigma_{\theta_0} (E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0} (E_1)) \bowtie_{\theta} E_2$$

b) When the selection condition θ_1 involves the attributes of E_1 only, and θ_2 includes the attributes of E_2 only.

$$\sigma_{\theta_1 \wedge \theta_2} (E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1} (E_1)) \bowtie_{\theta} (\sigma_{\theta_2} (E_2))$$

Rule 7: Distribution of the projection operation over the theta join.

Under two following conditions, the selection operation gets distributed over the theta-join operation:

a) Assume that the join condition θ includes only in $L_1 \cup L_2$ attributes of E_1 and E_2 Then, we get the following expression:

$$\pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = (\pi_{L_1} (E_1)) \bowtie_{\theta} (\pi_{L_2} (E_2))$$

b) Assume a join as $E_1 \bowtie_{\theta} E_2$. Both expressions E_1 and E_2 have sets of attributes as L_1 and L_2 . Assume two attributes L_3 and L_4 where L_3 be attributes of the expression E_1 , involved in the θ join condition but not in $L_1 \cup L_2$ Similarly, an L_4 be attributes of the expression E_2 involved only in the θ join condition and not in $L_1 \cup L_2$ attributes. Thus, we get the following expression:

$$\pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = \pi_{L_1 \cup L_2} ((\pi_{L_1 \cup L_3} (E_1)) \bowtie_{\theta} (\pi_{L_2 \cup L_4} (E_2)))$$

Rule 8: The union and intersection set operations are commutative.

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

However, set difference operations are not commutative.

Rule 9: The union and intersection set operations are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Rule 10: Distribution of selection operation on the intersection, union, and set difference operations.

The below expression shows the distribution performed over the set difference operation.

$$\sigma_p(E_1 - E_2) = \sigma_p(E_1) - \sigma_p(E_2)$$

We can similarly distribute the selection operation on $\cup$ and $\cap$ by replacing with $-$. Further, we get:

$$\sigma_p(E_1 - E_2) = \sigma_p(E_1) - E_2$$

Rule 11: Distribution of the projection operation over the union operation.

This rule states that we can distribute the projection operation on the union operation for the given expressions.

Resource utilization and performance

A typical transaction-processing application undergoes different demands throughout its various operating cycles. Peak loads during the day, week, month, and year, as well as the loads imposed by decision-support (DSS) queries or backup operations, can significantly impact any system that is running near capacity. You can use direct historical data derived from your particular system to pinpoint this impact.

You must take regular measurements of the workload and performance of your system to predict peak loads and compare performance measurements at different points in your usage cycle. Regular measurements help you to develop an overall performance profile for your database server applications. This profile is critical in determining how to improve performance reliably.

Utilization is the percentage of time that a component is actually occupied, as compared with the total time that the component is available for use. For instance, if a CPU processes transactions for a total of 40 seconds during a single minute, its utilization during that interval is 67 percent.

Measure and record utilization of the following system resources regularly:

- CPU
- Memory
- Disk

A resource is said to be *critical* to performance when it becomes overused or when its utilization is disproportionate to that of other components. For instance, you might consider a disk to be critical or overused when it has a utilization of 70 percent and all other disks on the system have 30 percent. Although 70 percent does not indicate that the disk is severely overused, you can improve performance by rearranging data to balance I/O requests across the entire set of disks.

How you measure resource utilization depends on the tools that your operating system provides for reporting system activity and resource utilization. After you identify a resource that seems overused, you can use the performance-monitoring utilities that the database server provides to gather data and make inferences about the database activities that might account for the load on that component. You can adjust your database server configuration or your operating system to reduce those database activities or spread them among other components. In some cases, you might need to provide additional hardware resources to resolve a performance bottleneck.

- ✓ **Resource utilization**

Whenever a system resource, such as a CPU or a particular disk, is occupied by a transaction or query, the resource is unavailable for processing other requests. Pending requests must wait for the resources to become available before they can complete.

- ✓ **CPU utilization**

Estimates of CPU utilization and response time can help you determine if you need to eliminate or reschedule some activities.

- ✓ **Memory utilization**

Memory is not managed as a single component, such as a CPU or disk, but as a collection of small components called *pages*.

- ✓ **Disk utilization**

because transfer rates vary among disks, most operating systems do not report disk utilization directly. Instead, they report the number of data transfers per second (in operating-system memory-page-size units.)

Query-Execution Plan in SQL

The steps of the process performed at the time of query execution by the engine database are described by a set of instructions called a query plan. The query plan is also referred to as the SQL Server execution plan.

The query optimizer generates the SQL Server execution plan or query plan. The generation of an optimum and economic query plan is the main objective of the query optimizer. Multiple execution plans are generated by the query processing engine after query execution, and from those generated execution plans, a plan with the best performance is selected. A plan cache is a memory location where the execution plans are stored and can be reused.

Execution plans are available in 3 forms in the SQL server, which are XML plans, Graphical plans, and Text plans.

Types of Execution Plan:

Estimated Execution Plan and Actual Execution Plan are two kinds of execution plans:

- 1. Actual Execution Plan:** When a query is executed then the Actual Execution Plan comes into the picture. At the time of query execution, the real process and strategy included are shown.
- 2. Estimated Execution Plan:** The query processor only guesses the precise actions involved at the time of the result returning. Sometimes, it is generated before the query execution.

Generating and Saving Execution Plans in SQL Server Management Studio:

Before and after the execution of the query, the execution plans in SQL Server. Actual and estimated execution plans can be achieved by the given steps:

Generation of Actual Execution Plans:

The actual execution plan can be achieved in the following ways in SQL Server:

1. After completely writing the query, Press Ctrl+M, and the actual execution plan will be generated.
2. Go to the query window and right-click on it, then click on the context menu and select '*Display Actual Execution Plan*'.
3. Or the '*Display Actual Execution Plan*' icon can be directly selected from the toolbar.

Generation of Estimated Execution Plans:

An estimated execution plan can be achieved using the following ways in SQL Server:

1. After completely writing the query, Press Ctrl+L, and the plan will be generated.
2. Go to the query window and right-click on it, then click on the context menu and select "*Display Estimated Execution Plan*".
3. Or the "*Display Estimated Execution Plan*" icon can be directly selected from the toolbar.

How to save a plan?

One has to save the query after interpreting the plan produced by the query. SQL Server Management Studio has an extension of ".sqlplan" for saving the plan in the system.

Steps to save an execution plan:

1. Go to the plan window and right-click.
2. Click on 'Save Execution Plan As'.
3. Click on the folder or location where you want to save the execution plan, then give the name to the plan and click on 'Save'.

Estimating Query Cost

Though a system can create multiple plans for a query, the chosen method should be the best of all. It can be done by comparing each possible plan in terms of their estimated cost. For calculating the net estimated cost of any plan, the cost of each operation within a plan should be determined and combined to get the net estimated cost of the query evaluation plan.

The cost estimation of a query evaluation plan is calculated in terms of various resources that include:

- Number of disk accesses
- Execution time taken by the CPU to execute a query
- Communication costs in distributed or parallel database systems.

To estimate the cost of a query evaluation plan, we use the number of blocks transferred from the disk, and the number of disks seeks. Suppose the disk has an average block access time of t_s seconds and takes an average of t_T seconds to transfer x data blocks. The block access time is the sum of disk seeks time and rotational latency. It performs S seeks then the time taken will be $b \cdot t_T + S \cdot t_s$ seconds. If $t_T = 0.1$ ms, $t_s = 4$ ms, the block size is 4 KB, and its transfer rate is 40 MB per second. With this, we can easily calculate the estimated cost of the given query evaluation plan.

Generally, for estimating the cost, we consider the worst case that could happen. The users assume that initially, the data is read from the disk only. But there must be a chance that the information is already present in the main memory. However, the users usually ignore this effect, and due to this, the actual cost of execution comes out less than the estimated value.

The response time, i.e., the time required to execute the plan, could be used for estimating the cost of the query evaluation plan. But due to the following reasons, it becomes difficult to calculate the response time without actually executing the query evaluation plan:

- When the query begins its execution, the response time becomes dependent on the contents stored in the buffer. But this information is difficult to retrieve when the query is in optimized mode, or it is not available also.
- When a system with multiple disks is present, the response time depends on an interrogation that in "what way accesses are distributed among the disks?". It is difficult to estimate without having detailed knowledge of the data layout present over the disk.
- Consequently, instead of minimizing the response time for any query evaluation plan, the optimizers find it better to reduce the total **resource consumption** of the query plan. Thus to estimate the cost of a query evaluation plan, it is good to minimize the resources used for accessing the disk or use of the extra resources.

Indexing in DBMS

- Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.
- The index is a type of data structure. It is used to locate and access the data in a database table quickly.

Index structure:

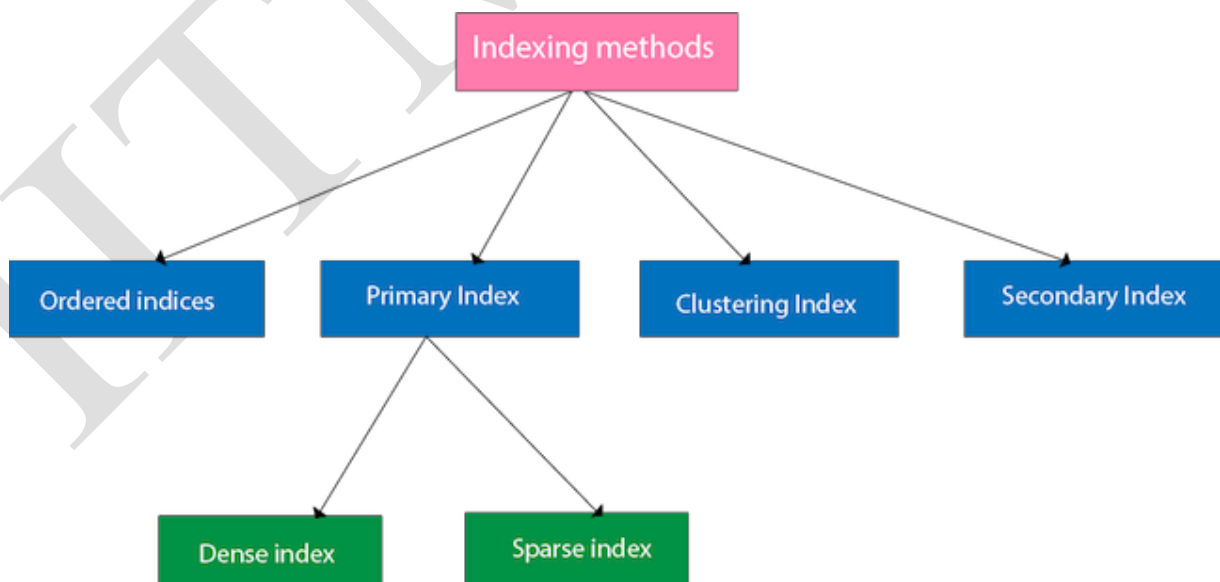
Indexes can be created using some database columns.

Search key	Data Reference
------------	----------------

Fig: Structure of Index

- The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Indexing Methods



Ordered indices

The indices are usually sorted to make searching faster. The indices which are sorted are known as ordered indices.

Example: Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDs start with 1, 2, 3....and so on and we have to search student with ID-543.

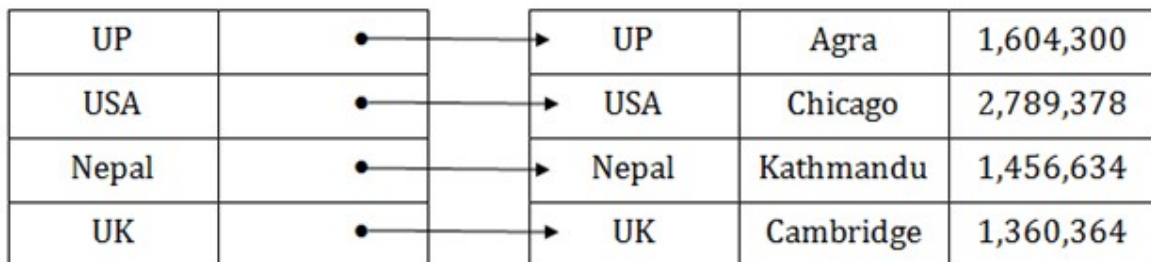
- In the case of a database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading $543 \times 10 = 5430$ bytes.
- In the case of an index, we will search using indexes and the DBMS will read the record after reading $542 \times 2 = 1084$ bytes which are very less compared to the previous case.

Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

Dense index

- The dense index contains an index record for every search key value in the data file. It makes searching faster.
- In this, the number of records in the index table is same as the number of records in the main table.
- It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.



The diagram illustrates a dense index. On the left is an index table with two columns: a search key column and a pointer column. On the right is a main data table with three columns: a search key column, a location column, and a value column. Arrows point from each row in the index table to the corresponding row in the main data table, demonstrating a one-to-one relationship.

UP	●	UP	Agra	1,604,300
USA	●	USA	Chicago	2,789,378
Nepal	●	Nepal	Kathmandu	1,456,634
UK	●	UK	Cambridge	1,360,364

Sparse index

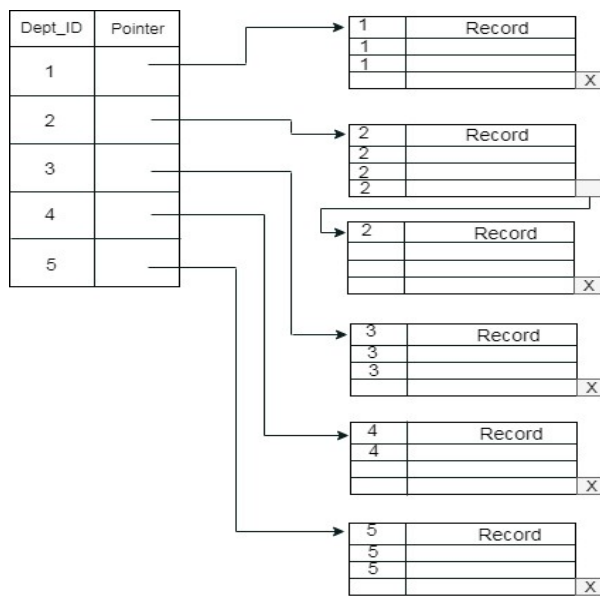
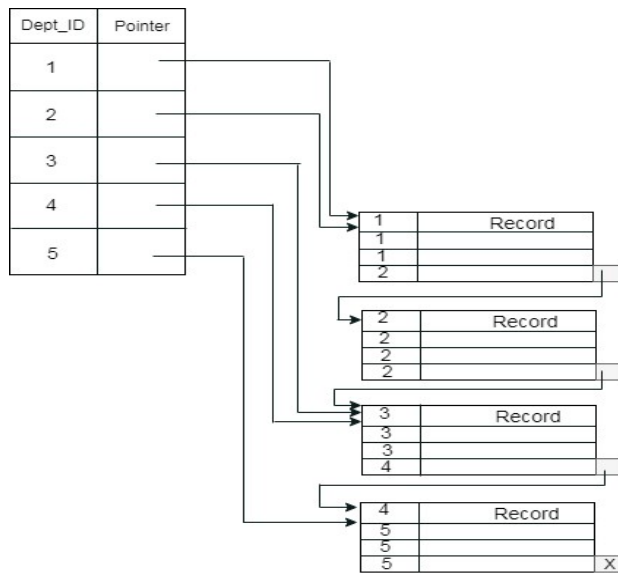
- In the data file, index record appears only for a few items. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.

UP	•	UP	Agra	1,604,300
Nepal	•	USA	Chicago	2,789,378
UK	•	Nepal	Kathmandu	1,456,634
		UK	Cambridge	1,360,364

Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

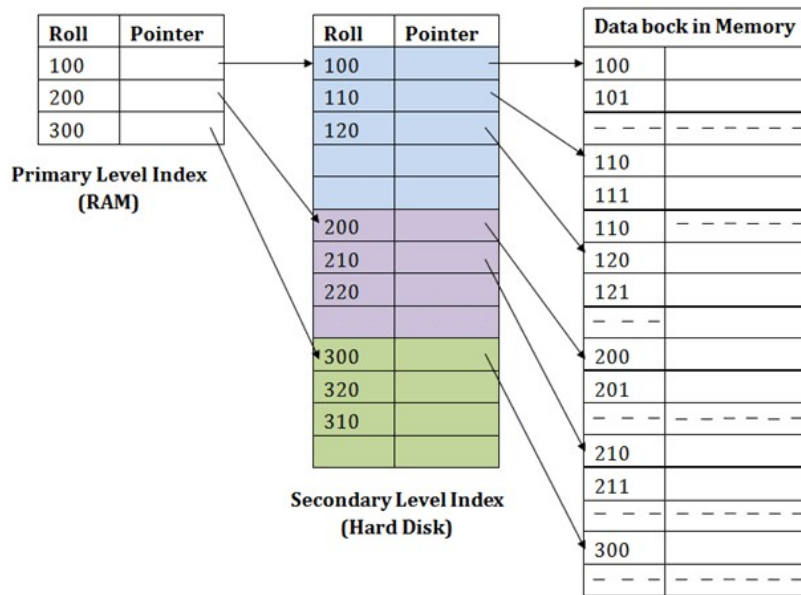
Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



Secondary Index

In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. Then the secondary memory searches the actual data based on the address got from mapping. If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, secondary indexing is introduced.

In secondary indexing, to reduce the size of mapping, another level of indexing is introduced. In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).



For example:

- If you want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level.
- Then in the second index level, again it does $\max(111) \leq 111$ and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111.
- This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.

SQL JOIN

As the name shows, JOIN means to combine something. In case of SQL, JOIN means "to combine two or more tables".

In SQL, JOIN clause is used to combine the records from two or more tables in a database.

Types of SQL JOIN

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN

1. INNER JOIN

In SQL, INNER JOIN selects records that have matching values in both tables as long as the condition is satisfied. It returns the combination of all rows from both the tables where the condition satisfies.

Syntax

1. SELECT table1.column1, table1.column2, table2.column1,....
2. FROM table1
3. INNER JOIN table2
4. ON table1.matching_column = table2.matching_column;

2. LEFT JOIN

The SQL left join returns all the values from left table and the matching values from the right table. If there is no matching join value, it will return NULL.

Syntax

1. SELECT table1.column1, table1.column2, table2.column1,....
2. FROM table1
3. LEFT JOIN table2
4. ON table1.matching_column = table2.matching_column;

3. RIGHT JOIN

In SQL, RIGHT JOIN returns all the values from the values from the rows of right table and the matched values from the left table. If there is no matching in both tables, it will return NULL.

Syntax

1. SELECT table1.column1, table1.column2, table2.column1,....
2. FROM table1
3. RIGHT JOIN table2
4. ON table1.matching_column = table2.matching_column;

4. FULL JOIN

In SQL, FULL JOIN is the result of a combination of both left and right outer join. Join tables have all the records from both tables. It puts NULL on the place of matches not found.

Syntax

1. SELECT table1.column1, table1.column2, table2.column1,....
2. FROM table1
3. FULL JOIN table2
4. ON table1.matching_column = table2.matching_column;

Join methods

The optimizer can choose one of three basic join strategies when queries require tables to be joined: nested-loop join, merge join, or hash join.

Nested-loop join

A nested-loop join is performed in one of the following two ways:

- Scanning the inner table for each accessed row of the outer table

For example, column A in table T1 and column A in table T2 have the following values:

Outer table T1: Column A	Inner table T2: Column A
2	3
3	2
3	2
	3
	1

To complete a nested-loop join between tables T1 and T2, the database manager performs the following steps:

1. Read the first row in T1. The value for A is 2.
2. Scan T2 until a match (2) is found, and then join the two rows.
3. Repeat Step 2 until the end of the table is reached.
4. Go back to T1 and read the next row (3).
5. Scan T2 (starting at the first row) until a match (3) is found, and then join the two rows.
6. Repeat Step 5 until the end of the table is reached.
7. Go back to T1 and read the next row (3).
8. Scan T2 as before, joining all rows that match (3).

Merge join

A merge join, sometimes known as a *merge scan join* or a *sort merge join*, requires a predicate of the form `table1.column = table2.column`. This is called an *equality join predicate*. A merge join requires ordered input on the joining columns, either through index access or by sorting. A merge join cannot be used if the join column is a LONG field column or a large object (LOB) column.

In a merge join, the joined tables are scanned at the same time. The outer table of the merge join is scanned only once. The inner table is also scanned once, unless repeated values occur in the outer table. If repeated values occur, a group of rows in the inner table might be scanned again.

For example, column A in table T1 and column A in table T2 have the following values:

Outer table T1: Column A	Inner table T2: Column A
2	1
3	2
3	2
	3
	3

To complete a merge join between tables T1 and T2, the database manager performs the following steps:

1. Read the first row in T1. The value for A is 2.
2. Scan T2 until a match (2) is found, and then join the two rows.
3. Keep scanning T2 while the columns match, joining rows.
4. When the 3 in T2 is read, go back to T1 and read the next row.
5. The next value in T1 is 3, which matches T2, so join the rows.
6. Keep scanning T2 while the columns match, joining rows.
7. When the end of T2 is reached, go back to T1 to get the next row. Note that the next value in T1 is the same as the previous value from T1, so T2 is scanned again, starting at the first 3 in T2. The database manager remembers this position.

Hash join

A hash join requires one or more predicates of the form `table1.columnX = table2.columnY`. None of the columns can be either a LONG field column or a LOB column.

A hash join is performed as follows: First, the designated inner table is scanned and rows are copied into memory buffers that are drawn from the sort heap specified by the **sortheap** database configuration parameter. The memory buffers are divided into sections, based on a hash value that is computed on the columns of the join predicates. If the size of the inner table exceeds the available sort heap space, buffers from selected sections are written to temporary tables.

When the inner table has been processed, the second (or outer) table is scanned and its rows are matched with rows from the inner table by first comparing the hash value that was computed for the columns of the join predicates. If the hash value for the outer row column matches the hash value for the inner row column, the actual join predicate column values are compared.

Outer table rows that correspond to portions of the table that are not written to a temporary table are matched immediately with inner table rows in memory. If the corresponding portion of the inner table was written to a temporary table, the outer row is also written to a temporary table. Finally, matching pairs of table portions from temporary tables are read, the hash values of their rows are matched, and the join predicates are checked.

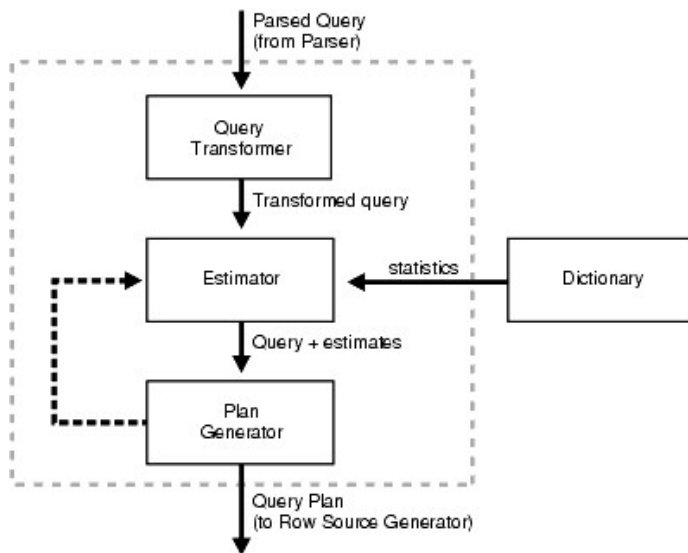
Hash join processing can also use filters to improve performance. If the optimizer chooses to use filters, and sufficient memory is available, filters can often reduce the number of outer table rows which need to be processed by the join.

For the full performance benefit of hash joins, you might need to change the value of the **sortheap** database configuration parameter and the **sheapthres** database manager configuration parameter.

Hash join performance is best if you can avoid hash loops and overflow to disk. To tune hash join performance, estimate the maximum amount of memory that is available for **sheapthres**, and then tune the **sortheap** parameter. Increase its setting until you avoid as many hash loops and disk overflows as possible, but do not reach the limit that is specified by the **sheapthres** parameter.

Increasing the **sortheap** value should also improve the performance of queries that have multiple sorts.

Structure of Query Optimizer



Components of the Query Optimizer

The query optimizer operations include:

- Query Transformation
- Estimation
- Plan Generation

Query Transformation

Each query portion of a statement is called a **query block**. The input to the query transformer is a parsed query, which is represented by a set of query blocks.

In the following example, the SQL statement consists of two query blocks. The subquery in parentheses is the **inner query block**. The **outer query block**, which is the rest of the SQL statement, retrieves names of employees in the departments whose IDs were supplied by the subquery.

```
SELECT first_name, last_name
FROM employees
WHERE department_id
IN (SELECT department_id FROM departments WHERE location_id = 1800);
```

The query form determines how query blocks are interrelated. The transformer determines whether it is advantageous to rewrite the original SQL statement into a semantically equivalent SQL statement that can be processed more efficiently.

The query transformer employs several query transformation techniques, including the following:

- View Merging
- Predicate Pushing
- Subquery Unnesting
- Query Rewrite with Materialized Views

Any combination of these transformations can apply to a given query.

View Merging

Each view referenced in a query is expanded by the parser into a separate query block. The block essentially represents the view definition, and thus the result of a view. One option for the optimizer is to analyze the view query block separately and generate a view subplan. The optimizer then processes the rest of the query by using the view subplan to generate an overall query plan. This technique usually leads to a suboptimal query plan because the view is optimized separately.

Predicate Pushing

In **predicate pushing**, the optimizer "pushes" the relevant predicates from the containing query block into the view query block. For views that are not merged, this technique improves the subplan of the unmerged view because the database can use the pushed-in predicates to access indexes or to use as filters.

Subquery Unnesting

In **subquery unnesting**, the optimizer transforms a nested query into an equivalent join statement, and then optimizes the join. This transformation enables the optimizer to take advantage of the join optimizer technique. The optimizer can perform this transformation only if the resulting join statement is guaranteed to return exactly the same rows as the original statement, and if subqueries do not contain aggregate functions such as **AVG**.

Query Rewrite with Materialized Views

A **materialized view** is like a query with a result that the database materializes and stores in a table. When the database finds a user query compatible with the query associated with a materialized view, then the database can rewrite the query in terms of the materialized view. This technique improves query execution because most of the query result has been precomputed.

The query transformer looks for any materialized views that are compatible with the user query and selects one or more materialized views to rewrite the user query. The use of materialized views to rewrite a query is cost-based. That is, the optimizer does not rewrite the query if the plan generated without the materialized views has a lower cost than the plan generated with the materialized views.

Estimation

The **estimator** determines the overall cost of a given execution plan. The estimator generates three different types of measures to achieve this goal:

- Selectivity

This measure represents a fraction of rows from a row set. The selectivity is tied to a query predicate, such as `last_name='Smith'`, or a combination of predicates.

- Cardinality

This measure represents the number of rows in a row set.

- Cost

This measure represents units of work or resource used. The query optimizer uses disk I/O, CPU usage, and memory usage as units of work.

If statistics are available, then the estimator uses them to compute the measures. The statistics improve the degree of accuracy of the measures.

Selectivity

The **selectivity** represents a fraction of rows from a row set. The row set can be a base table, a view, or the result of a join or a GROUP BY operator. The selectivity is tied to a query predicate, such as `last_name = 'Smith'`, or a combination of predicates, such as `last_name = 'Smith' AND job_type = 'Clerk'`.

A predicate filters a specific number of rows from a row set. Thus, the selectivity of a predicate indicates how many rows pass the predicate test. Selectivity ranges from 0.0 to 1.0. A selectivity of 0.0 means that no rows are selected from a row set, whereas a selectivity of 1.0 means that all rows are selected. A predicate becomes more selective as the value approaches 0.0 and less selective (or more unselective) as the value approaches 1.0.

The optimizer estimates selectivity depending on whether statistics are available:

- Statistics not available

Depending on the value of the `OPTIMIZER_DYNAMIC_SAMPLING` initialization parameter, the optimizer either uses **dynamic statistics** or an internal default value. The database uses different internal defaults depending on the predicate type. For

example, the internal default for an equality predicate (`last_name = 'Smith'`) is lower than for a range predicate (`last_name > 'Smith'`) because an equality predicate is expected to return a smaller fraction of rows. See ["Controlling Dynamic Statistics"](#).

- Statistics available

When statistics are available, the estimator uses them to estimate selectivity. Assume there are 150 distinct employee last names. For an equality predicate `last_name = 'Smith'`, selectivity is the reciprocal of the number n of distinct values of `last_name`, which in this example is .006 because the query selects rows that contain 1 out of 150 distinct values.

If a histogram is available on the `last_name` column, then the estimator uses the histogram instead of the number of distinct values. The histogram captures the distribution of different values in a column, so it yields better selectivity estimates, especially for columns that contain **skewed data**. See ["Viewing Histograms"](#).

Cardinality

Cardinality represents the number of rows in a row set. In this context, the row set can be a base table, a view, or the result of a join or GROUP BY operator.

Cost

The **cost** represents units of work or resource used in an operation. The optimizer uses disk I/O, CPU usage, and memory usage as units of work. The operation can be scanning a table, accessing rows from a table by using an index, joining two tables together, or sorting a row set. The cost is the number of work units expected to be incurred when the database executes the query and produces its result.

The **access path** determines the number of units of work required to get data from a base table. The access path can be a table scan, a fast full index scan, or an index scan.

- Table scan or fast full index scan

During a table scan or **fast full index scan**, the database reads multiple blocks from disk in a single I/O. Therefore, the cost of the scan depends on the number of blocks to be scanned and the multiblock read count value.

- Index scan

The cost of an index scan depends on the levels in the B-tree, the number of index leaf blocks to be scanned, and the number of rows to be fetched using the rowid in the index keys. The cost of fetching rows using rowids depends on the **index clustering factor**. See ["Assessing I/O for Blocks, not Rows"](#).

The **join cost** represents the combination of the individual access costs of the two row sets being joined, plus the cost of the join operation.

Plan Generation

The **plan generator** explores various plans for a query block by trying out different access paths, join methods, and join orders. Many plans are possible because of the various combinations of different access paths, join methods, and join orders that the database can use to produce the same result. The purpose of the generator is to pick the plan with the lowest cost.

Join Order

A **join order** is the order in which different join items, such as tables, are accessed and joined together. Assume that the database joins **table1**, **table2**, and **table3**. The join order might be as follows:

1. The database accesses **table1**.
2. The database accesses **table2** and joins its rows to **table1**.
3. The database accesses **table3** and joins its data to the result of the join between **table1** and **table2**.

Query Subplans

The optimizer represents each nested subquery or unmerged view by a separate query block and generates a **subplan**. The database optimizes query blocks separately from the bottom up. Thus, the database optimizes the innermost query block first and generates a subplan for it, and then lastly generates the outer query block representing the entire query.

The number of possible plans for a query block is proportional to the number of join items in the **FROM** clause. This number rises exponentially with the number of join items. For example, the possible plans for a join of five tables will be significantly higher than the possible plans for a join of two tables.

Cutoff for Plan Selection

The plan generator uses an internal cutoff to reduce the number of plans it tries when finding the lowest-cost plan. The cutoff is based on the cost of the current best plan. If the current best cost is large, then the plan generator explores alternative plans to find a lower cost plan. If the current best cost is small, then the generator ends the search swiftly because further cost improvement will not be significant.

The cutoff works well if the plan generator starts with an initial join order that produces a plan with cost close to optimal. Finding a good initial join order is a difficult problem.